

MXEN3005 Robot Manipulation Project
2026 Semester 1

Design Project Handout
ROS2 package for teleoperation of Dobot Magician

Ask your tutor to check the hardware before powering Dobot Magician for the first time.

Make sure Dobot Magician workspace is clean and empty before commencing work.

Never attach/wire anything to Dobot Magician while it is powered on.

Always shutdown Dobot Magician before attaching/wiring anything to it.

1. Introduction

This handout document provides the description and example use of the Keyboard class that allows for checking if a key is up or down.

Important note: the Keyboard class uses evdev module that detects key presses even if the terminal that runs the code is out of focus. For this reason, one of the required functionalities in the Design Project is to implement a toggle key (keypad asterisk *) that enables/disables keyboard control so that Dobot Magician will not move unintentionally.

2. Using the Keyboard class

The Python file “keyboard.py” that implements the class Keyboard is provided on Blackboard. To use the class, you must import it and create the corresponding object. This object then provides a single function, called “.read_key()” that returns a tuple with two elements. The meaning of the elements is described below.

- If a key has been pressed or released since the last call to the function, then the first element of the tuple is the key code in the format “KEY_<name>” where <name> is the text representation of the key, and the second element is either equal to 1 if the key has been pressed or equal to 0 if the key has been released. For example, after pressing “W” key the “.read_key()” function will return (‘KEY_W’, 1) and after releasing the key it will return (‘KEY_W’, 0).
- If no key has been pressed or released since the last call to the function, then the tuple returned is (None, None).

Important note: you need to install the corresponding Python package to use the evdev module:

```
sudo apt update  
sudo apt install python3-evdev
```

An example of a Python file that monitors key presses at the rate of 100 Hz is provided on Blackboard under the name “example1.py” and pasted below.

```
from keyboard import Keyboard
import time

kb = Keyboard()

# monitor key presses at 100 Hz
while True:
    key = kb.read_key()
    if not key == (None, None):
        print(f"{key}")

    time.sleep(0.01)
```

Running this file (“python3 example1.py”) and then pressing “A”, pressing “W”, releasing “W” and finally releasing “A” will print the following to the terminal:

```
('KEY_A', 1)
('KEY_W', 1)
('KEY_W', 0)
('KEY_A', 0)
```

If you hold the Enter key for too long when running the above example file, then the first output will actually be ('KEY_ENTER', 0) showing that the release of the Enter key was also detected. Another example of a Python file that monitors if a specific key was pressed and/or released at the same 100 Hz rate is provided on Blackboard under the name “example2.py” and pasted below.

```
from keyboard import Keyboard
import time

kb = Keyboard()

# monitor key “A” presses at 100 Hz
while True:
    key = kb.read_key()
    if key == ('KEY_A', 1):
        print(f"Do something when key A is pressed")

    if key == ('KEY_A', 0):
        print(f"Do something when key A is released")

    time.sleep(0.01)
```

After running this file (“python3 example2.py”), try pressing and then releasing the “A” key. The following output will be printed to the terminal:

```
('KEY_A', 1)
('KEY_A', 0)
```

However, pressing and releasing any other key will be ignored and nothing will be printed to the terminal.

Important note: do not blindly copy the infinite loop (“while True:”) to the file that implements a ROS2 node. You must instead use the ROS2 functionality for performing repeating tasks (hint: use appropriate callback function).